

Optimización de aprendizaje dinámico en conmutadores Ethernet de alto rendimiento

Trabajo Fin de Grado

Ingeniería en Tecnologías de Telecomunicación

Luis Galindo Ortuño

Granada, septiembre de 2026

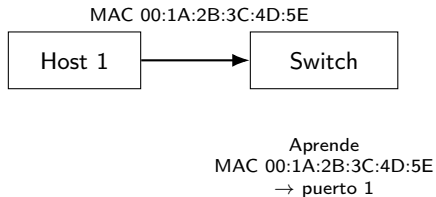
Índice

- ▶ Introducción
- ▶ Diseño de las posibles soluciones
- ▶ Validación funcional en Python
- ▶ Implementación en HDL
- ▶ Implementación en FPGA
- ▶ Evaluación de resultados
- ▶ Conclusiones

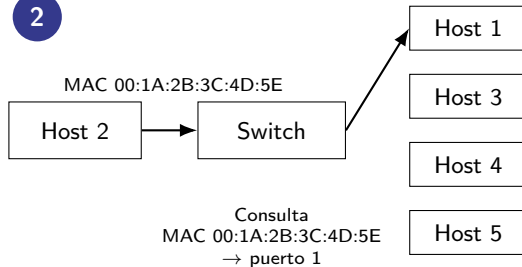
Introducción

Aprendizaje dinámico de direcciones MAC

1



2



- La tabla MAC puede recibir más peticiones de escritura de las que puede procesar.

Objetivos

Objetivo general

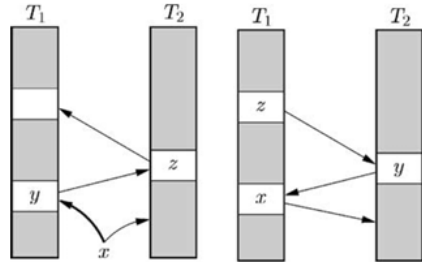
Diseñar, implementar e integrar una solución hardware sobre FPGA para la optimización del aprendizaje dinámico de la tabla MAC en conmutadores Ethernet de alto rendimiento.

Objetivos secundarios

- ▶ Revisión del estado del arte.
- ▶ Modelado del tráfico Ethernet.
- ▶ Diseño de las posibles soluciones.
- ▶ Validación funcional en Python.
- ▶ Implementación en HDL.
- ▶ Implementación en FPGA.
- ▶ Validación y evaluación de la solución.

Cuckoo Hashing

- ▶ Para insertar el elemento x se calculan sus posiciones candidatas y se intenta almacenar en alguna de ellas.
- ▶ En caso de que estén ocupadas, se intentan recolocar.



Ref : R. Pagh and F. F. Rodler, "Cuckoo Hashing," Journal of Algorithms, 2004.

FPGA

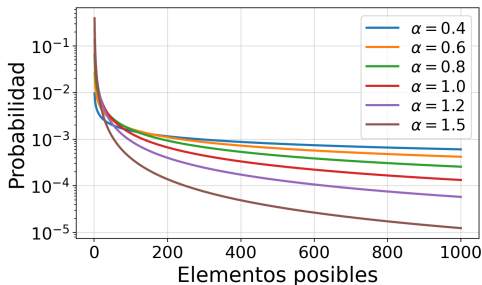


Placa de evaluación ZCU102 con dispositivo Zynq UltraScale+ MPSoC XCZU9EG

Ref : AMD, "ZCU102 Evaluation Board."

Modelado del tráfico Ethernet: Distribución de Zipf

$$P(i) = \frac{1}{i^\alpha} \frac{1}{\sum_{j=1}^N \frac{1}{j^\alpha}}$$



- **Breslau et al.:** observaron que la popularidad de contenidos en sistemas de caché Web puede aproximarse mediante una distribución de Zipf.
- N : número total de elementos posibles; i : posición en el ranking de popularidad; α : controla el grado de concentración del tráfico.

Ref : L. Breslau et al., "Web Caching and Zipf-like Distributions: Evidence and Implications," 1999.

Metodología de diseño

- ▶ **Implementación en Python:** validación funcional de los modelos propuestos.
- ▶ **Implementación en HDL:** validar que el modelo propuesto puede describirse como una arquitectura RTL sintetizable
- ▶ **Implementación en FPGA:** validar que el diseño puede sintetizarse, implementarse y ejecutarse en hardware real con un consumo de recursos reducido.

Diseño de las posibles soluciones

TinyLFU

Política de admisión basada en frecuencia reciente.

TinyLFU utiliza una estimación de la frecuencia reciente utilizando un Count-Min Sketch (CMS) para decidir qué elementos deben ser admitidos en la caché.

Proceso de admisión:

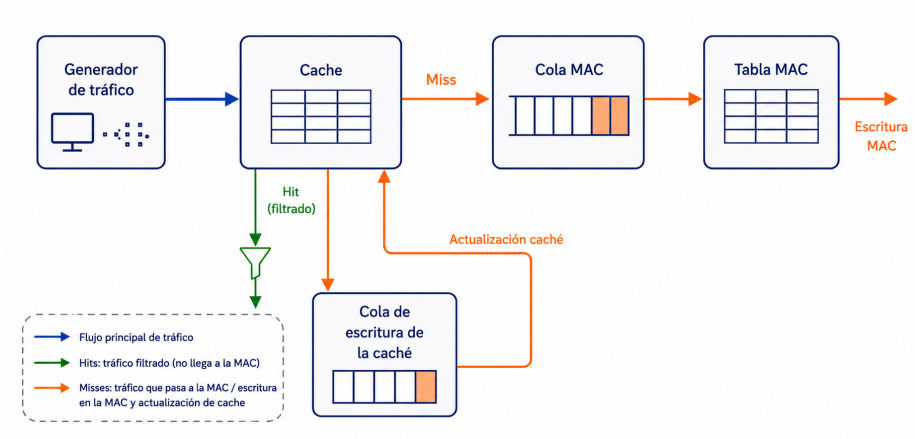
1. Se estima la frecuencia del objeto entrante.
2. Se estima la frecuencia del candidato a ser desalojado.
3. Se comparan ambas frecuencias.
4. Se admite únicamente el elemento con mayor frecuencia estimada.

Aplicación en este trabajo: se toma TinyLFU como punto de partida para el diseño de las soluciones de filtrado propuestas.

Ref : G. Einziger and R. Friedman, "TinyLFU: A Highly Efficient Cache Admission Policy," 2014.

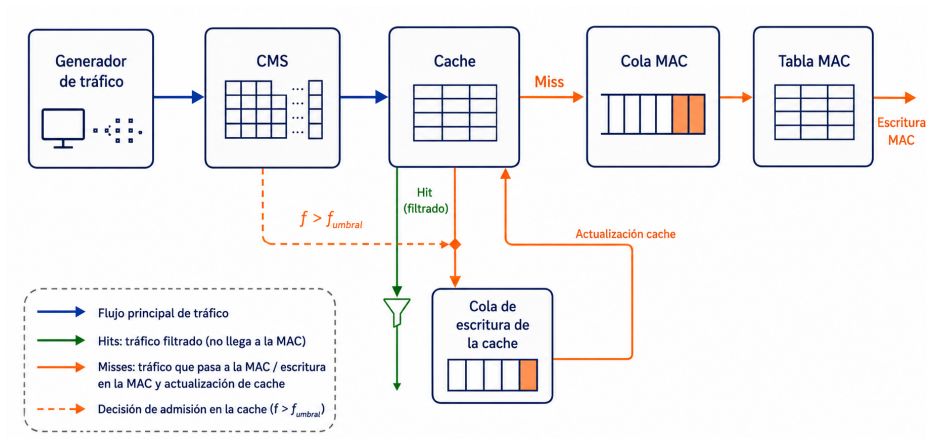
Soluciones propuestas

Modelo con caché



Soluciones propuestas

Modelo CMS + caché



Métricas de evaluación

Porcentaje de filtrado de la caché

$$P_{\text{filtrado}} = \frac{N_{\text{hits}}}{N_{\text{hits}} + N_{\text{misses}}} \cdot 100$$

Porcentaje de aprendizaje

$$P_{\text{aprendizaje}} = \frac{N_{\text{MAC aprendidas}}}{N_{\text{MAC generadas}}} \cdot 100$$

Porcentaje de descartes por saturación en la tabla MAC

$$P_{\text{descartes}} = \frac{N_{\text{descartes MAC}}}{N_{\text{tramas generadas}}} \cdot 100$$

Eficiencia de escritura

$$P_{\text{eficiencia}} = \frac{N_{\text{escrituras útiles}}}{N_{\text{intentos de escritura}}} \cdot 100$$

Validación funcional en Python

Herramientas de modelado

Librería SimPy: librería de Python para simulación de eventos discretos.

- ▶ **Environment:** gestiona el tiempo y los eventos de la simulación.
- ▶ **process():** permite ejecutar procesos concurrentes.
- ▶ **timeout():** modela el avance del tiempo y las latencias del sistema.

Módulo Hash_Table.py

- ▶ Se utiliza como bloque de memoria para implementar la caché y la tabla MAC.
- ▶ Implementa Cuckoo Hashing para las operaciones de lectura y escritura.

Ref : C. Megías et al., "Minimizing Cuckoo Hashing Insertion Time for Networking Applications in FPGAs," IEEE Access, 2025.

Implementación de los bloques

► Generador de tráfico

- Distribución de Zipf.

► Tabla MAC

- 16384 posiciones y 4 tablas paralelas.
- Escritor separado del pipeline principal.

► Caché

- 4096 entradas y 4 tablas paralelas.
- Escritor separado del pipeline principal.

► CMS

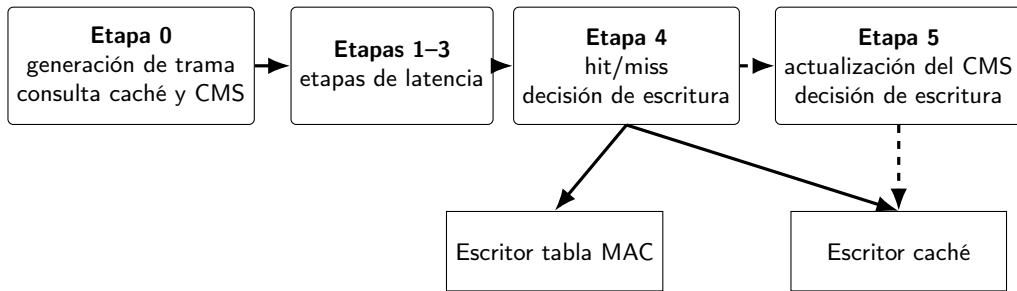
- 4 filas y 2048 columnas.
- Umbral de admisión igual a 2.
- Se activa a partir del 90 % de ocupación de la caché.

► Colas de caché y MAC

- FIFO con 64 elementos.

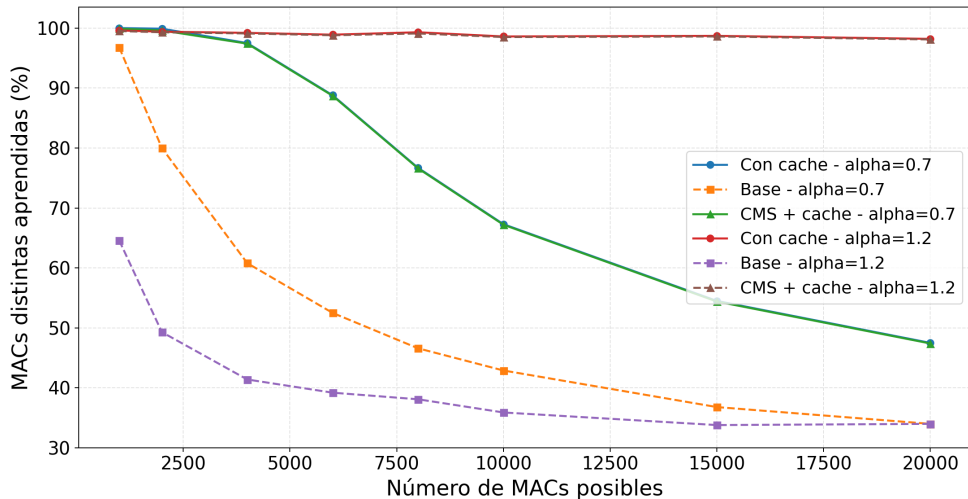
Implementación del Pipeline

---> modelo CMS + caché.



Validación del modelo

Porcentaje de direcciones MAC aprendidas a 51,2 Gbps



Implementación en HDL

Herramientas del modelado HDL

Descripción y simulación

- ▶ **Verilog HDL**: descripción RTL sintetizable.
- ▶ **cocotb**: banco de pruebas en Python.
- ▶ **GTKWave**: visualización y análisis de señales.

Módulos proporcionados

- ▶ `hash_table.v`: bloque de memoria Cuckoo Hashing.
- ▶ `switch_simple_fifo.v`: bloque de memoria para implementar colas FIFO.

Implementación de los bloques HDL

► Caché

- 4096 entradas y 4 tablas paralelas.
- Implementada mediante instancia de `hash_table.v`.

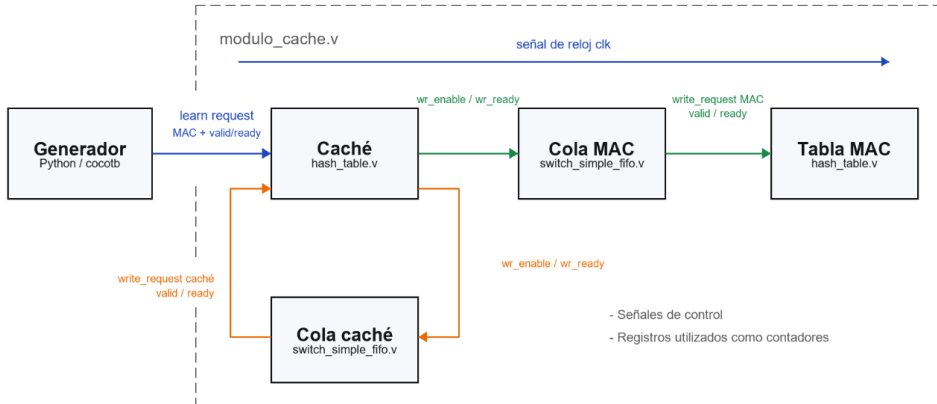
► Tabla MAC

- 16384 entradas y 4 tablas paralelas.
- Implementada mediante instancia de `hash_table.v`.

► Colas de escritura caché y tabla MAC

- Colas FIFO con 64 posiciones.
- Implementadas mediante instancia de `switch_simple_fifo.v`.

Implementación arquitectura HDL



Implementación en FPGA

Herramientas e integración en FPGA

Herramientas

- ▶ **Vivado**: síntesis, implementación y generación del bitstream.

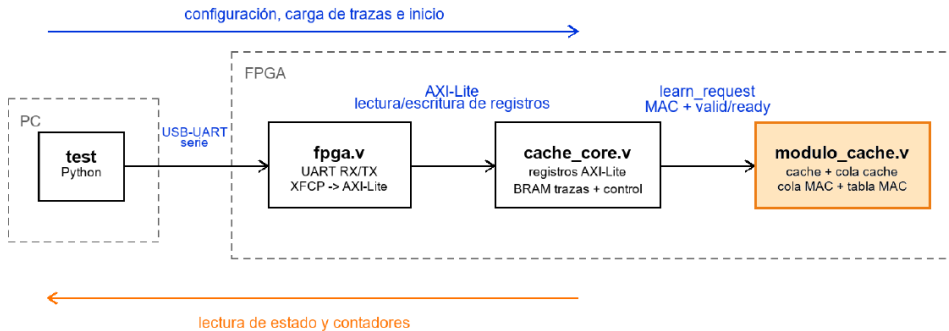
Protocolos de comunicación

- ▶ **USB-UART**: comunicación serie entre el PC y la FPGA.
- ▶ **XFCP y AXI4-Lite**: acceso a los registros internos del diseño.

Módulos proporcionados

- ▶ `fpga.v`: nivel superior de la arquitectura.
- ▶ `fpga_core.v`: base para el desarrollo de `cache_core.v`.

Implementación arquitectura FPGA



Evaluación de resultados

Validación cruzada y consumo de recursos

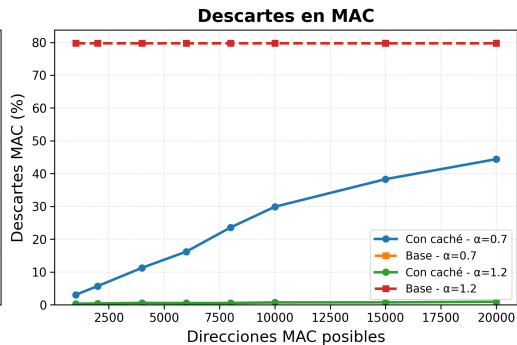
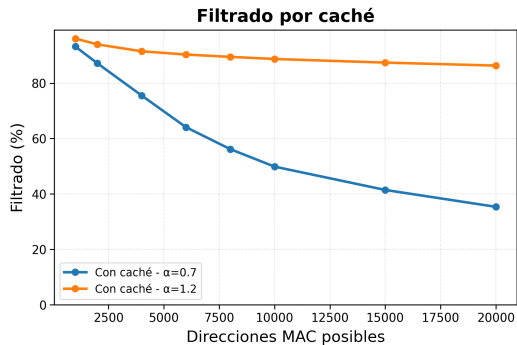
Validación cruzada

Métrica	Py-HDL	HDL-FPGA
Filtrado	0,48 %	0,00 %
Descartes	0,47 %	0,00 %
Aprendizaje	0,26 %	0,00 %
Eficiencia	0,88 %	0,00 %

Consumo de recursos FPGA XCZU9EG

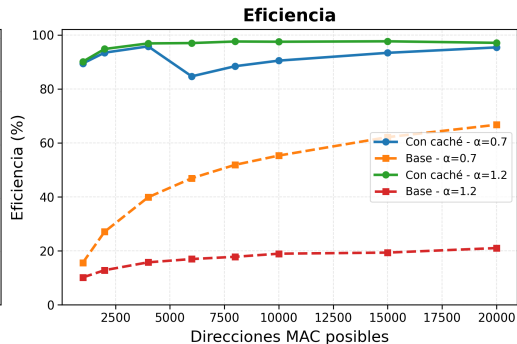
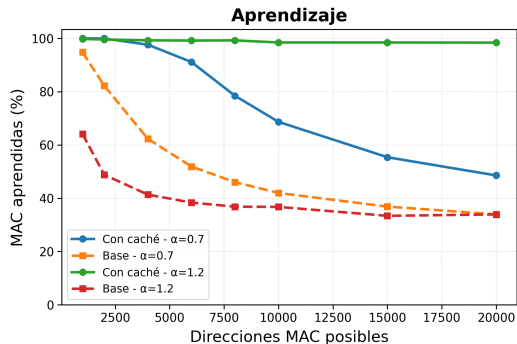
Recurso	Utilizado	Disponible	Uso
CLB LUTs	3701	274080	1,35 %
CLB Registers	2934	548160	0,54 %
Block RAM Tile	125	912	13,71 %
DSPs	0	2520	0,00 %

Barrido muestral de direcciones MAC a 51,2 Gbps



► La caché reduce la presión sobre la cola de la tabla MAC.

Barrido muestral de direcciones MAC a 51,2 Gbps



- Al reducir escrituras redundantes, aumenta la proporción de operaciones útiles.

Conclusiones

Aportación: reducir las escrituras redundantes de la tabla MAC con un consumo de recursos reducido mejorando ampliamente su funcionamiento.

Objetivos: se han cumplido todos los objetivos planteados al principio del proyecto.

Trabajo futuro: integración en la arquitectura completa de un switch Ethernet y evaluación con tráfico real.

Se incluye un repositorio en Github con todo el material del trabajo :

<https://github.com/luisgalindoort/>

Optimizacion-aprendizaje-dinamico-conmutadores-Ethernet-alto-rendimiento.